

VIII JORNADAS

STIC CCN-CERT

La defensa del patrimonio tecnológico
frente a los ciberataques

10 y 11 de diciembre de 2014



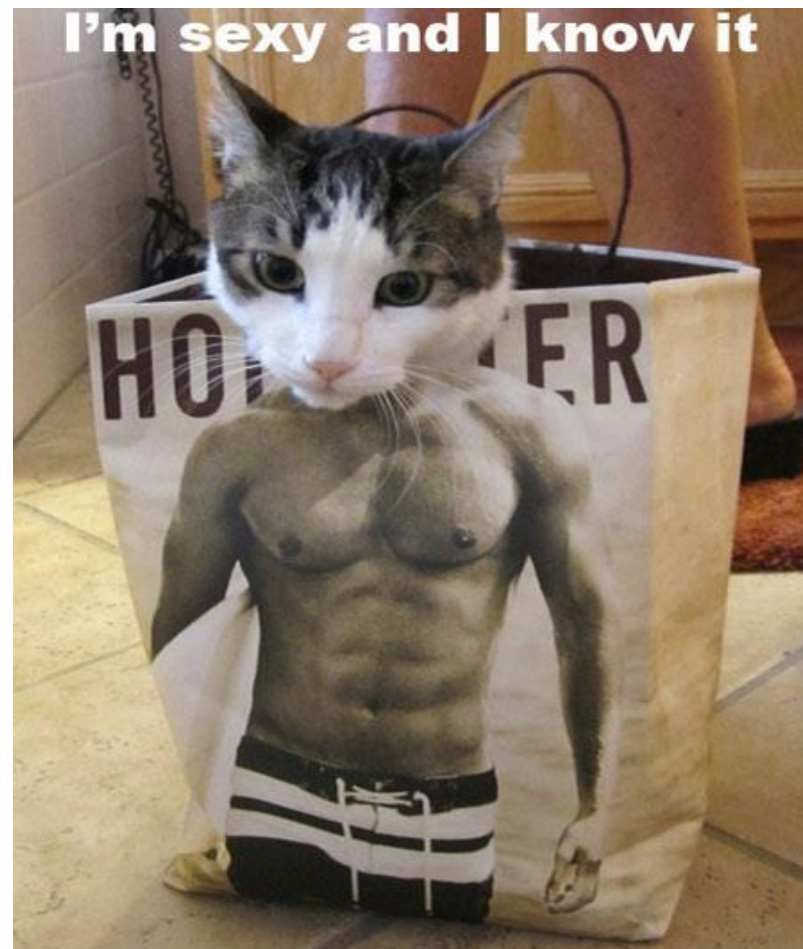
UEFI: Un arma de doble filo



BIOS



UEFI



1.1 BIOS / UEFI. ¿Dónde estamos?

- Mucho parque informático aún con sistemas de arranque basados en BIOS.
- UEFI actualmente evoluciona gracias a la aportación de un grupo de empresas OEMs, IBVs, ISVs, IHVs.
- A pesar de la irrupción de UEFI, para las plataformas x86 siempre habrá elementos de código BIOS presentes. (CSM).
- En ambos mundos, siempre habrá unas cuantas líneas en ensamblador, para inicializar el HW al menos.
- Reducido número de fabricantes de BIOS (IBV)
 - Ventaja para el atacante



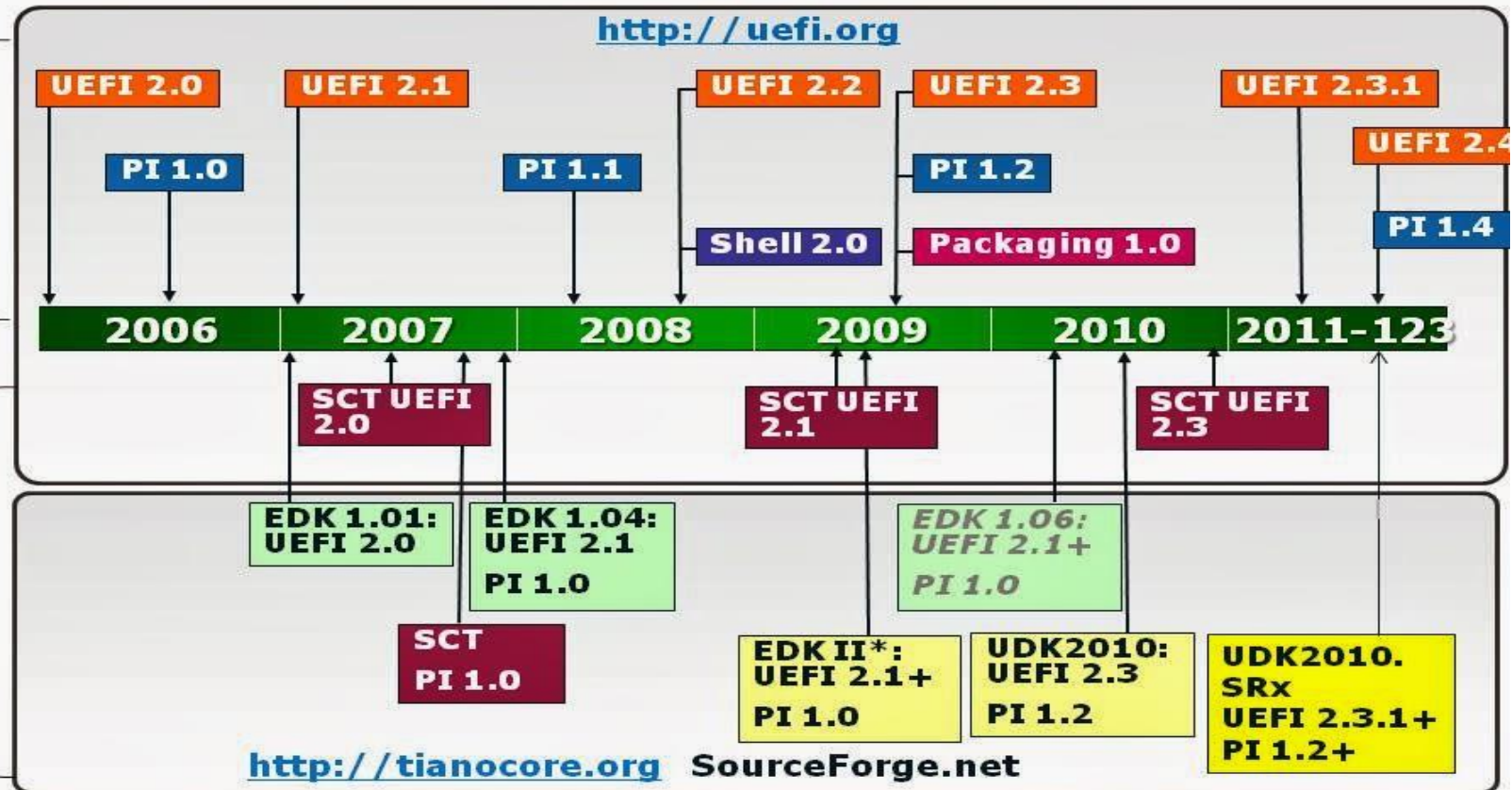
UEFI – Linea de tiempos

Specification & Tianocore.org Timeline

<http://uefi.org>

Specifications

Implementation

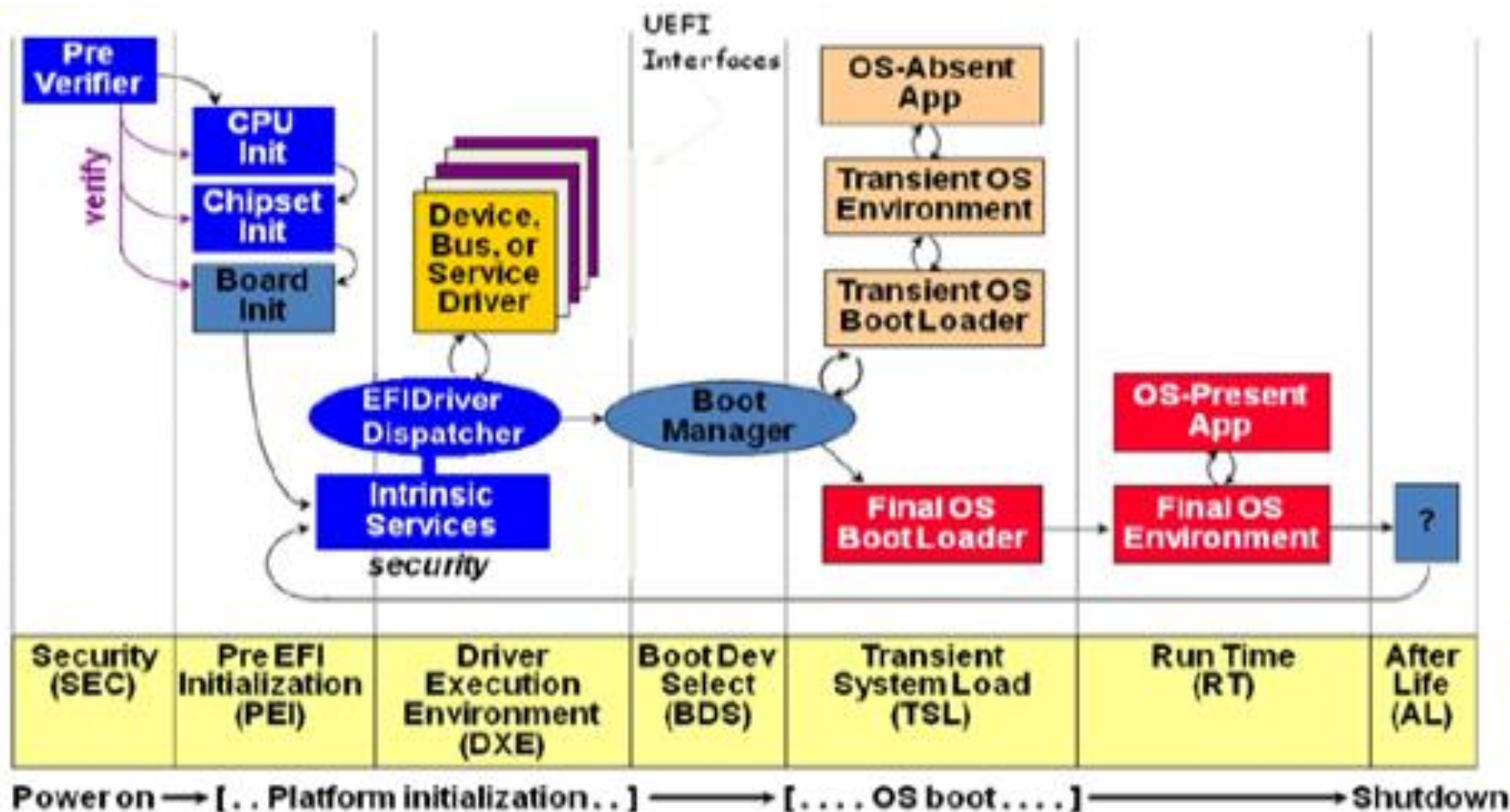


All products, dates, and programs are based on current expectations and subject to change without notice.

UEFI. ¿Qué es?

- UEFI - Es una especificación de una interfaz para interactuar con el firmware de una plataforma a la hora de arrancar un sistema.
 - PI – (Platform Initialization) define el cómo se hace esta inicialización.
- UEFI es una especificación independiente de la plataforma
 - PI – Aglutina la parte dependiente de la arquitectura específica
- Participan en el Unified EFI Forum más de 240 empresas
 - AMD, AML, Apple, Dell, HP, IBM, Insyde, Intel, Lenovo, Microsoft, and Phoenix Technologies (PROMOTORES).
- Permite una estrategia de generación de código compartida entre OEMs, IBVs, IHVs
- Crecimiento en volumen de código enorme
 - HP Elitebook 2540p (201?): 42 PEIMs, 164 DXE drivers
 - HP Elitebook 850 G1 (2014): 117 PEIMs, 392 DXE drivers

UEFI – Fases diferenciadas en el arranque



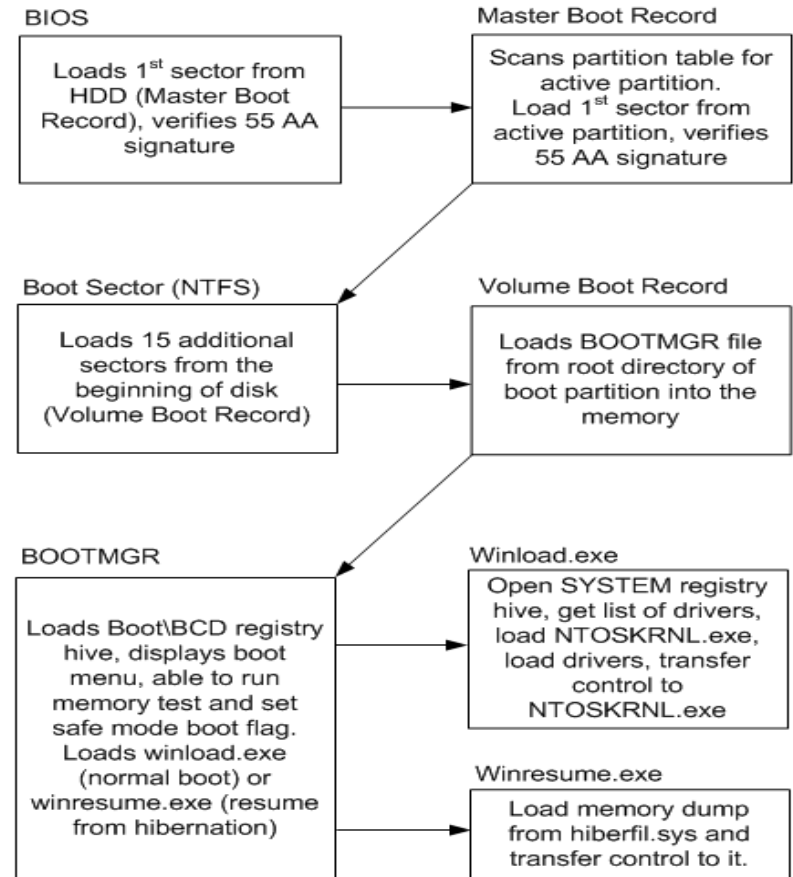
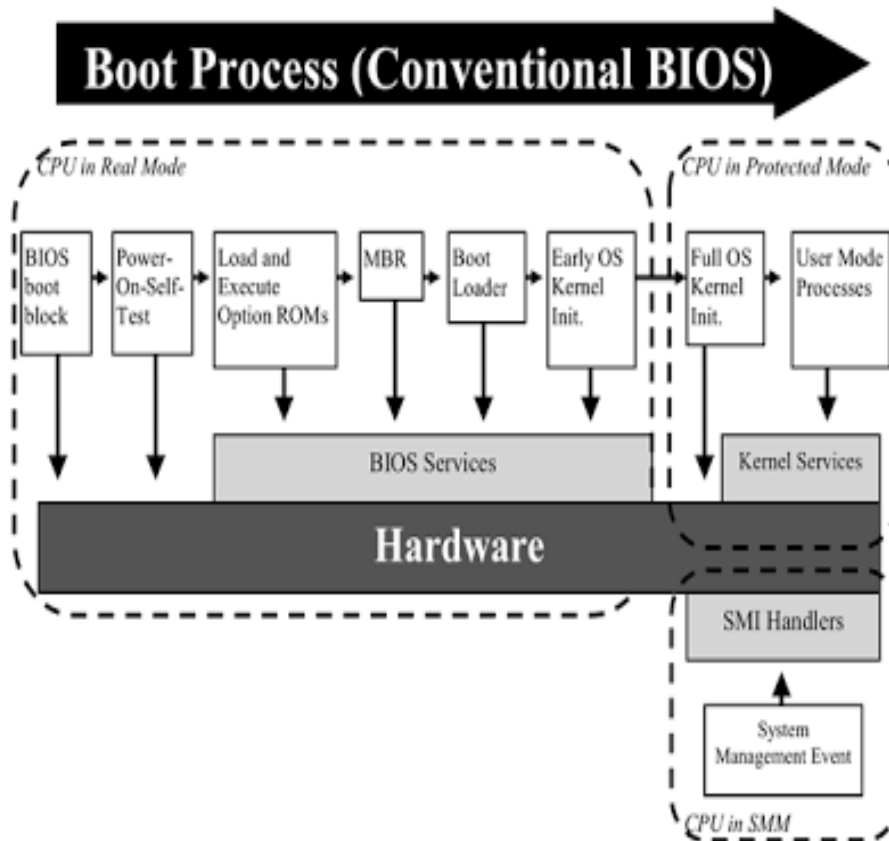
UEFI en un flash (1)

- UEFI es una especificación independiente de la arquitectura
 - Existe tanto para plataformas de 32 y 64 bits
 - Actualmente soporta: ITANIUM, x86, x64, ARM (32/64), EBC.
- PI (Platform Initialization) aglutina la parte dependiente de la arquitectura
- Modelo formal de extensibilidad de la arquitectura
 - Uso de Firmware Volumes (FVs), Firmware File Systems (FFSs)
 - GUIDs,
 - Formato PE en los ejecutables,
 - Análisis de dependencias de módulos.
- Parte del código típicamente reside en FW, parte en soporte externo.
 - EFI system Partition (ESP). Mínimo 200 Mb y formateada como FAT32
 - Variables BCD se almacenan típicamente en la ESP
 - Requiere disco duro particionado como GPT

UEFI en un flash (2)

- UEFI corre en long-mode en x64
 - Entorno óptimo para utilizar técnicas modernas de programación y herramientas
 - BIOS por el contrario es un entorno de 16-bits en modo real
- Complementa el Advanced Configuration and Power Interface (ACPI).
 - Diferentes alternativas de código según el tipo de arranque → Mayor superficie de ataque
- Run-time Services limitados en UEFI de cara al S.O.
 - Básicamente Set/Get Variables que permiten por ejemplo alterar el proceso de arranque.
- Soporta protocolos IPv4 e IPv6 para la localización de módulos de arranque por red
 - También se puede arrancar por red (PXE).

Proceso de arranque en BIOS



Proceso de arranque en UEFI

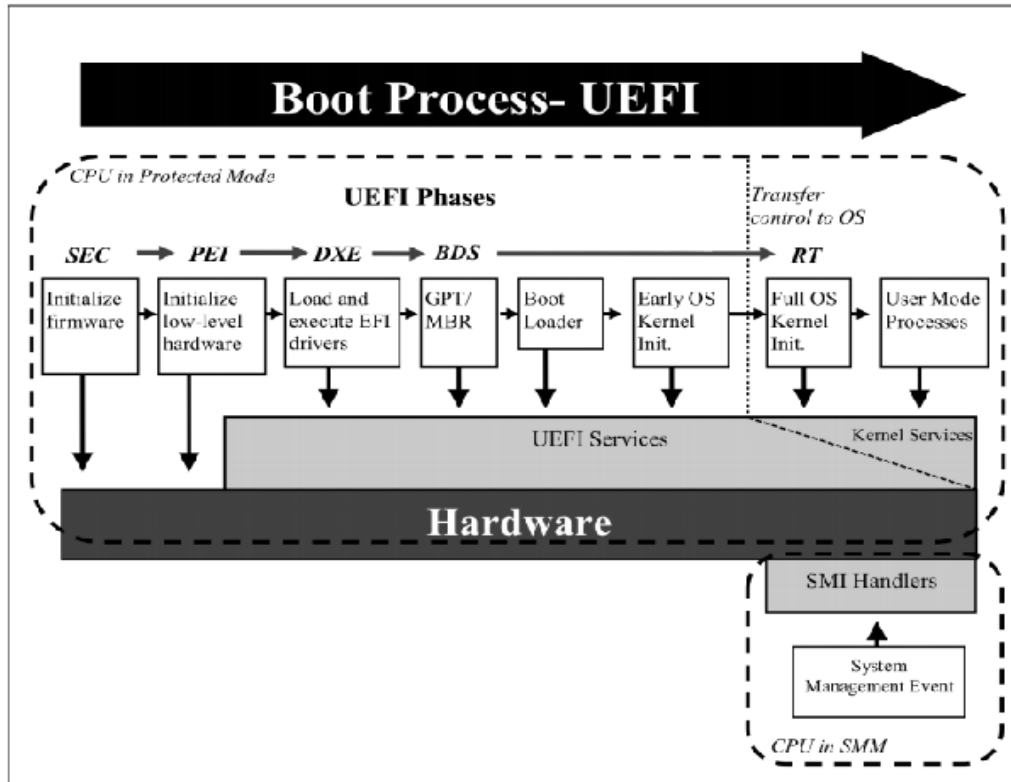
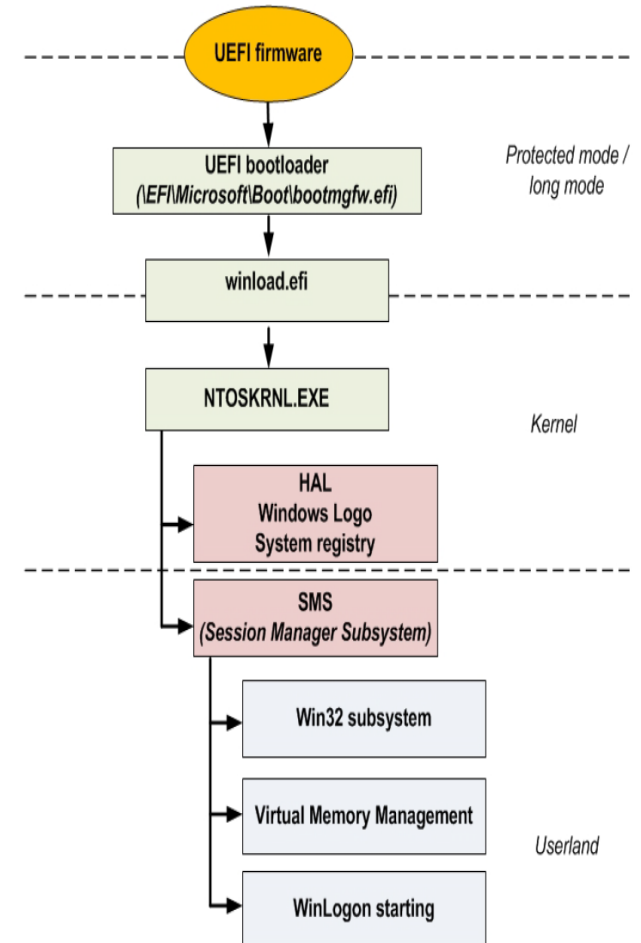


Figure 2: UEFI BIOS Boot Process

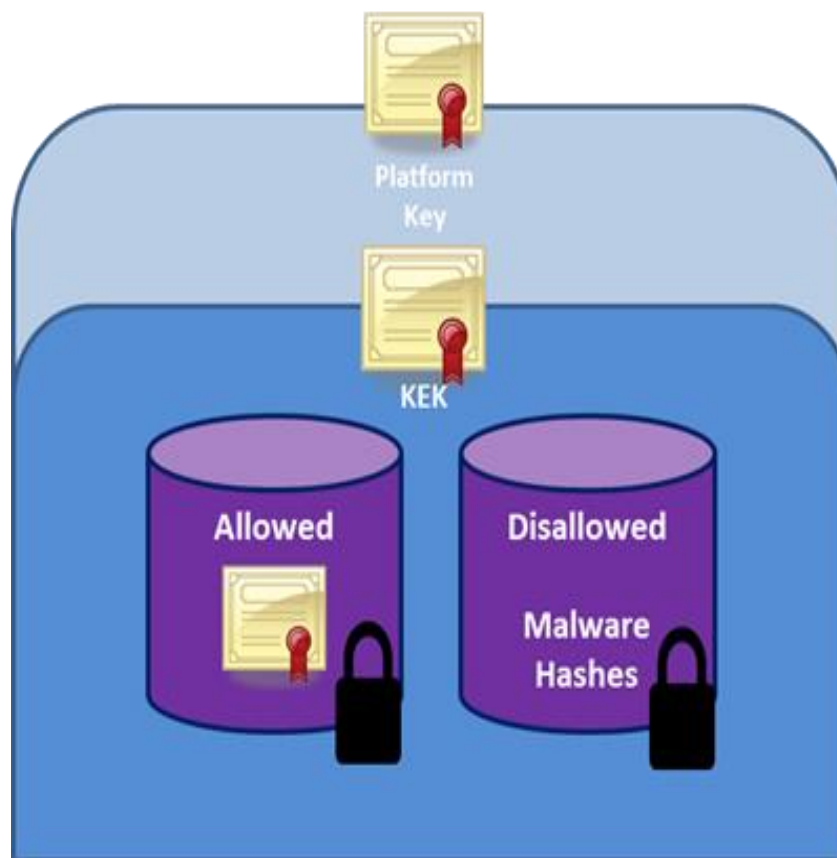


SECURE BOOT -

- Desde la especificación UEFI 2.3.1 se añadió como característica de seguridad para proteger los sistemas UEFI de Bootkits
- Valida la integridad del cargador de arranque del Sistema Operativo antes de transferir el control.
- No requiere de TPM (Trusted Platform Module)
- Cuando está habilitado se debe inhabilitar el CSM (legacy boot)
- Se apoya en una jerarquía de claves con una responsabilidad compartida.
- En plataformas Microsoft, funciona para Windows 8.x y 2012 Server.

SECURE BOOT - Claves

- PK – Relación de confianza entre el dueño de la plataforma y el FW
- KEKs – Entre el FW y el/los OSs
- DB – firmas o hashes de módulos permitidos
- DBx – firmas o hashes de módulos no permitidos
- Se verifican cuando el BootLoader carga la imagen



Listado de Claves en Menú BIOS

Security	
Manage All Factory Keys (PK,KEK,DB,DBX) Install default Secure Boot keys	
Platform Key (PK)	NOT INSTALLED
▶ Set PK from File ▶ Get PK to File ▶ Delete the PK	
Key Exchange Key Database(KEK)	INSTALLED
▶ Set KEK from File ▶ Get KEK to File ▶ Delete the KEK ▶ Append an entry to KEK	
Authorized Signature Database(DB)	INSTALLED
▶ Set DB from File ▶ Get DB to File ▶ Delete the DB ▶ Append an entry to DB	
Forbidden Signature Database(DBX)	INSTALLED
▶ Set DBX from File ▶ Get DBX to File ▶ Delete the DBX ▶ Append an entry to DBX	
Force System to User Mode - install default Secure Boot Variables(PK,KEK,db,dx). Change takes effect after reboot	
→← : Select Screen ↑↓ : Select Item Enter: Select +/- : Change Opt. F1 : General Help F9 : Optimized Defaults F10 : Save & Exit ESC : Exit	

Powershell – Verificación del estado de las claves

```

Administrator: Windows PowerShell

PS C:\Windows\system32> Get-SecureBootPolicy

Publisher                                     Version
-----
77fa9abd-0359-4d32-bd60-28f4e78f784b         1

PS C:\Windows\system32> Get-SecureBootUEFI -Name SecureBoot | Format-List

Name      : SecureBoot
Bytes     : {1}
Attributes : BOOTSERVICE_ACCESS
            RUNTIME_ACCESS
            AUTHENTICATED WRITE ACCESS

PS C:\Windows\system32> Get-SecureBootUEFI -Name SetupMode | Format-List

Name      : SetupMode
Bytes     : {0}
Attributes : BOOTSERVICE_ACCESS
            RUNTIME_ACCESS
            AUTHENTICATED WRITE ACCESS

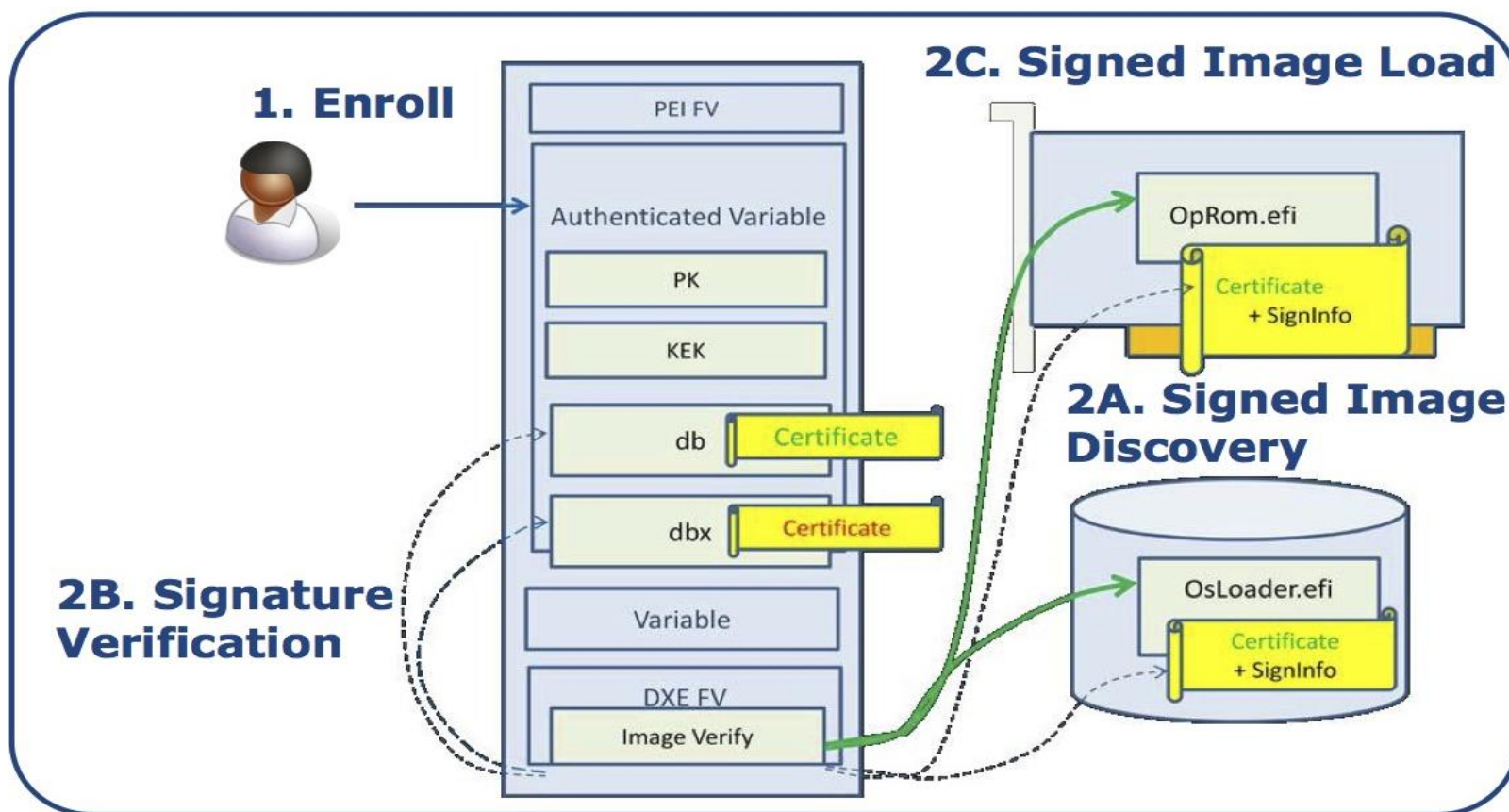
PS C:\Windows\system32> Get-SecureBootUEFI -Name PK | Format-List

Name      : PK
Bytes     : {161, 89, 192, 165...}
Attributes : NON_VOLATILE
            BOOTSERVICE_ACCESS
            RUNTIME_ACCESS
            TIME BASED AUTHENTICATED WRITE ACCESS
  
```

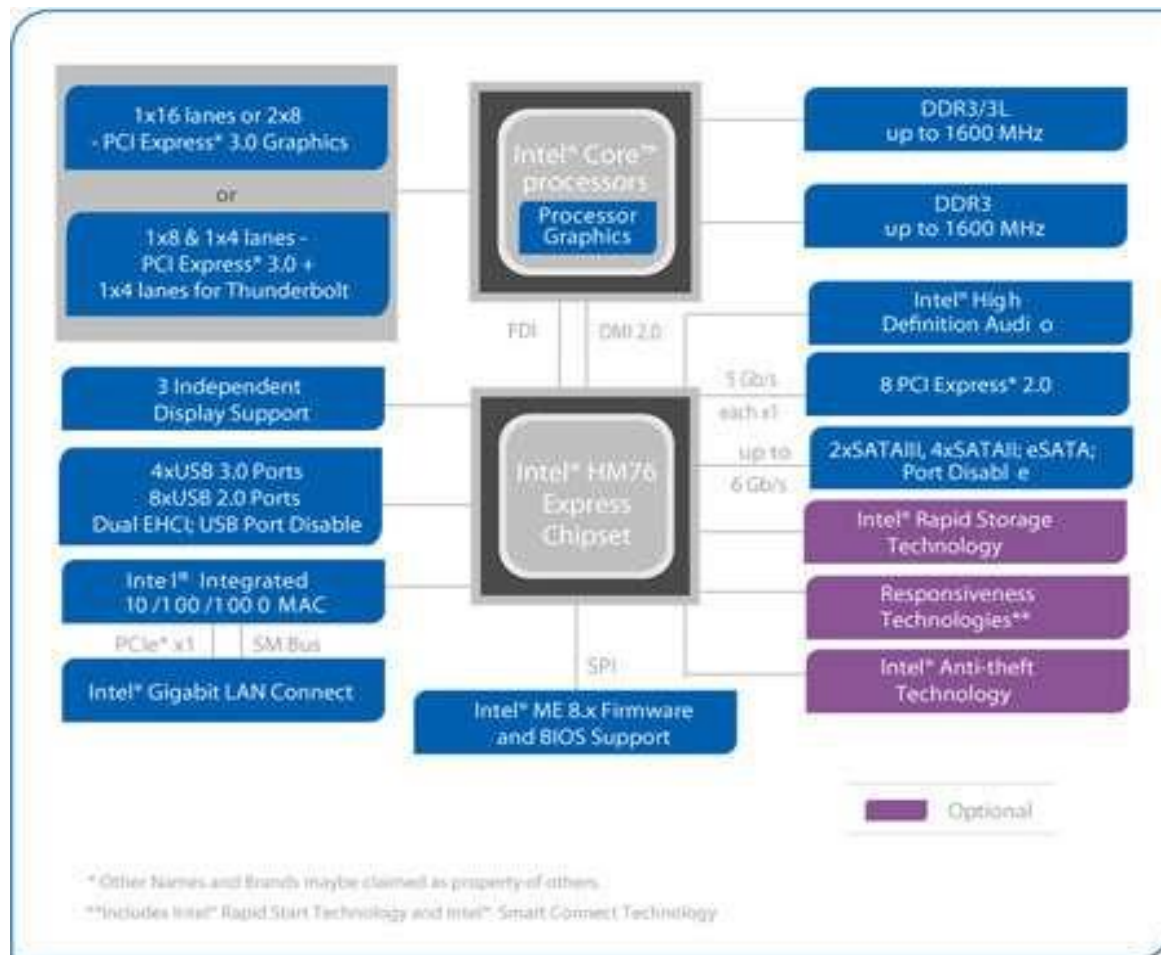
- Get/Set-SecureBootUEFI Get-SecureBootPolicy
- Confirm/Format-SecureBootUEFI

Proceso de verificación de firmas en SecureBoot

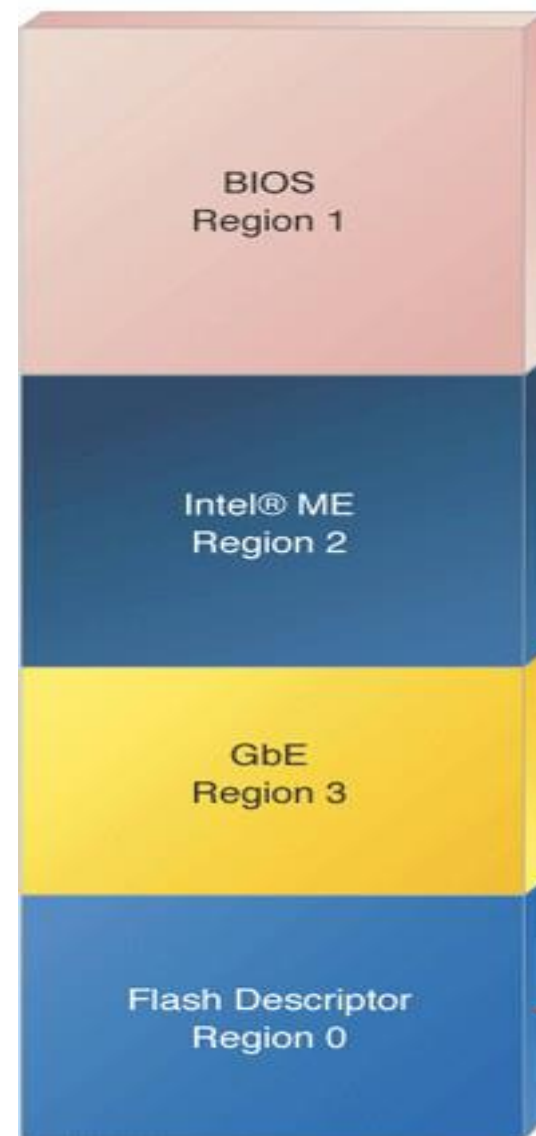
- Realizada por el Boot Manager
- Una vez insertado el hash o firma, se da por bueno.



NVRAM. Ubicación y contenidos.



Intel® HM76 Express Chipset Platform Block Diagram



Fptw64 - Volcado a disco de la FLASH

- Depende de la configuración de variables NVRAM

```
Administrador: cmd.exe

D:\platforms\asus\Firmware Programming Tool\Windows64>fptw64 -I

Intel (R) Flash Programming Tool. Version: 8.1.40.1456
Copyright (c) 2007 - 2013, Intel Corporation. All rights reserved.

Platform: Intel(R) HM76 Express Chipset
Reading HSFSTS register... Flash Descriptor: Valid

--- Flash Devices Found ---
MX25L6405D      ID:0xC22017      Size: 8192KB (65536Kb)

--- Flash Image Information --
Signature: VALID
Number of Flash Components: 1
Component 1 - 8192KB (65536Kb)
Regions:
Descriptor - Base: 0x000000, Limit: 0x000FFF
BIOS        - Base: 0x200000, Limit: 0x7FFFFFFF
ME          - Base: 0x003000, Limit: 0x1FFFFFFF
GbE         - Base: 0x001000, Limit: 0x002FFF
PDR         - Not present
Master Region Access:
CPU/BIOS    - ID: 0x0000, Read: 0xFF, Write: 0xFF
ME          - ID: 0x0000, Read: 0xFF, Write: 0xFF
GbE         - ID: 0x0118, Read: 0xFF, Write: 0xFF

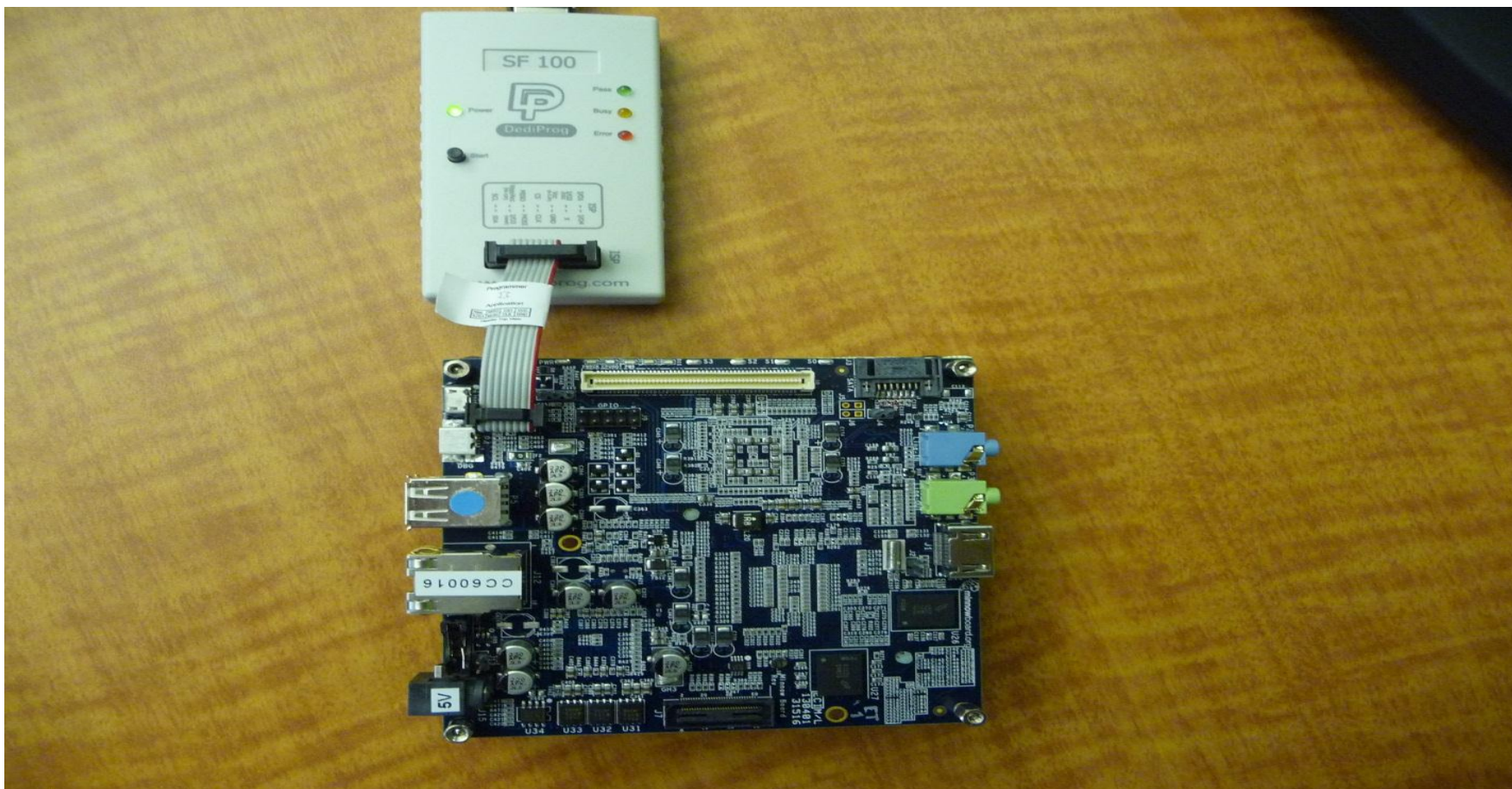
Total Accessable SPI Memory: 8192KB, Total Installed SPI Memory : 8192KB

FPT Operation Passed

D:\platforms\asus\Firmware Programming Tool\Windows64>
D:\platforms\asus\Firmware Programming Tool\Windows64>
```


Grabado de la NVRAM vía HW

- Esta opción siempre es viable.



UEFITOOL – Contenidos de volumen BIOS en UEFI

Volumen PEI – PEIMs.

UEFITool 0.19.4 - UX32VDAS.211

Name	Action	Type	Subtype	Text
AMI Aptio capsule		Capsule	AMI Aptio	
BIOS image		Image	BIOS	
Padding		Padding	Nonempty	
8C8CE578-8A3D-4F1C-9935-896185C32DD3		Volume		
CEF589A3-476D-497F-9FDC-E98143E0422C		File	Raw	
Padding		Padding	Empty(0xFF)	
8C8CE578-8A3D-4F1C-9935-896185C32DD3		Volume		
8C8CE578-8A3D-4F1C-9935-896185C32DD3		Volume		
3B42EF57-16D3-44CB-8632-9FDB06B41451		File	Boot	
PEI dependency section		Section	PEI module	MemoryInit
PE32+ image section		Section	PE32+ image	
User interface section		Section	User interface	
E008B434-0E73-440C-8612-A143F6A07BCB		File	PEI module	Recovery
0D1ED2F7-E928-4562-92DD-5C82EC917EAE		File	PEI module	CRBPEI
C41E9862-D078-4E7D-9062-00E3FAC34C19		File	PEI module	ASUSITEPei
1D88C542-9DF7-424A-AA90-02B61F286938		File	PEI module	WdtPei
92685943-D810-47FF-A112-CC8490776A1F		File	PEI core	CORE_PEI
01359D99-9446-456D-ADA4-50A711C03ADA		File	PEI module	CpuInitPei
C866BD71-7C79-4BF1-A938-066883008F9A		File	PEI module	CpuS3Peim
8B8214F9-4ADB-47DD-AC62-8313C537E9FA		File	PEI module	SmmBasePeim
0AC2D35D-1C77-1033-A6F8-7CA55DF7D0AA		File	PEI module	CpuPolicyPei
1555ACF3-BD07-4685-B668-A86945A4124D		File	PEI module	CpuPeiBeforeMem
2B85AFA9-FF33-417B-8497-CB773C2B93BF		File	PEI module	CpuPei
C1FBD624-27EA-40D1-AA48-94C3DC5C7E0D		File	PEI module	SBPEI
333BB2A3-4F20-4C8B-AC38-0672D74315F8		File	PEI module	AcpiPlatformPei
9EA28D33-0175-4788-BEA8-6950516030A5		File	PEI module	SmbusPei
0F69F6D7-0E48-43A6-BFC2-6871694369B0		File	PEI module	WdtAppPei
FD236AE7-0791-48C4-B29E-298DDEE1A838		File	PEI module	PchInitPeim
FF259F16-18D1-4298-8DD2-BD87FF2B94A9		File	PEI module	PchResetPeim
643DF777-F312-42ED-81CC-1B1F57E18AD6		File	PEI module	PchSmbusArpDisabled
AA652CB9-2D52-4624-9FAE-D4E58867CA46		File	PEI module	PchSpiPeim
6B4FDBD2-47E1-4A09-BA8E-8E041F208B95		File	PEI module	PchUsb

Information

Type: 0x1B
Size: 0x0000D8
Parsed expression:
PUSH 30EB2979-B0F7-4D60-B2DC-1A2C96CEB1F4
AND PUSH B6EC423C-21D2-490D-85C6-DD5864EAA674
AND PUSH 7408D748-FC8C-4EE6-9288-C4BEC092A410
AND PUSH 3CDC90C6-13FB-4A75-9E79-59E9DD78B9FA
AND PUSH 9A7EF41E-C140-4BD1-B884-1E11240B4CE6
AND PUSH ABD42895-78CF-4872-8444-1B5C180BFBDA
AND PUSH 150CE416-EE63-46B6-8BA3-7322B8E04637
AND PUSH F38D1338-AF7A-4FB6-91DB-1A9C2183570D
AND PUSH 8C376010-2400-4D7D-B478-9D851DF3C9D1
AND PUSH EE0EA811-FBD9-4777-B95A-BA4F71101F74
AND PUSH 3D0E663A-DC72-4489-87C5-E49EE773A452
AND PUSH 09EA8911-BE0D-4230-A003-EDC693B48E11
AND END

Messages

UEFITOOL – Contenidos de volumen BIOS en UEFI

Volumen DXE – Drivers.

UEFITool 0.19.4 - UX32VDAS.211

Name	Action	Type	Subtype	Text
AMI Aptio capsule		Capsule	AMI Aptio	
BIOS image		Image	BIOS	
Padding		Padding	Nonempty	
8C8CE578-8A3D-4F1C-9935-896185C32DD3		Volume		
CE5B9A3-476D-497F-9FDC-E98143E0422C		File	Raw	
Padding		Padding	Empty(0xFF)	
8C8CE578-8A3D-4F1C-9935-896185C32DD3		Volume		
5C266089-E103-4D43-9AB5-12D7095BE2AF		File	DXE driver	IntelSaGopDriver
5BBA83E6-F027-4CA7-BFD0-16358CC9E123		File	DXE driver	IntelIvbGopDriver
PE32+ image section		Section	PE32+ image	
User interface section		Section	User interface	
8D59EBC8-B85E-400E-970A-1F995D1DB91E		File	DXE driver	IntelSnbGopDriver
17088572-377F-44EF-8F4E-B09FF46A070		File	Raw	
E03ABADF-E536-4E88-83A0-B77F78E834FE		File	DXE driver	CpuDxe
08A2CA63-3B65-472C-874E-5E138E947324		File	DXE driver	ASUSITERT
A3EAA83C-BA3A-4524-9DC7-7E339996F496		File	DXE driver	ASUSRT
A3BC19A6-3572-4AF4-BCE4-CD43A8D1F6AF		File	DXE driver	ASUSITEBS
93022F8C-1F09-47EF-88B2-5814FF609DF5		File	DXE driver	FileSystem
DAC2B117-B5FB-4964-A312-0DCC77061B9B		File	Freeform	
9221315B-30BB-46B5-813E-1B1BF4712BD3		File	Freeform	
5AE3F37E-4EAE-41AE-8240-35465B5E81EB		File	DXE core	CORE_DXE
CBC59C4A-383A-41EB-A8EE-4498AE567E4		File	DXE driver	Runtime
3C1DE39F-D207-408A-AACC-731CFB7F1DD7		File	DXE driver	PciBus
9F3A0016-AE55-4288-829D-D22FD344C347		File	DXE driver	AmiBoardInfo
62D171CB-78CD-4480-8678-C6A2A797A8DE		File	DXE driver	CpuInitDxe
7FED72EE-0170-4814-9878-A8F81864DFAF		File	DXE driver	SmmRelocDxe
ABB74F50-FD2D-4072-A321-CAFC7297EFA		File	PEI module	SmmRelocPeim
5552575A-7E00-4D61-A3A4-F7547351B49E		File	DXE driver	SmmBaseRuntime
9CC55D7D-FBFF-431C-BC14-334EAE6052B		File	DXE driver	SmmDisp
8D3BE215-D6F6-4264-BEA6-28073FB13AEA		File	DXE driver	SmmThunk
15B9B6DA-00A9-4DE7-B8E8-ED7AF88F16E		File	DXE driver	CpuPolicyDxe
F3331DE6-4A55-44E4-B767-74537A1A021		File	DXE driver	MicrocodeUpdate

Information

Type: 0x15
Size: 0x000024
Text: IntelIvbGopDriver

Messages

UEFITOOL – Contenidos de volumen BIOS en UEFI

Capacidad de edición y sustitución de módulos.

UEFITool 0.19.4 - UX32VDAS.211

Action	Type	Subtype	Text
Section	Section	PEI dependency	MemoryInit
Section	Section	PE32+ image	
Section	Section	User interface	
File	File	PEI module	Recovery
File	File	PEI module	CRBPEI
File	File	PEI module	ASUSITEPei
File	File	PEI module	WdtPei
File	File	PEI core	CORE_PEI
File	File	PEI module	CpuInitPei
File	File	PEI module	CpuS3Peim
File	File	PEI module	SmmBasePeim
File	File	PEI module	CpuPolicyPei
File	File	PEI module	CpuPeiBeforeMem
File	File	PEI module	CpuPei
File	File	PEI module	SBPEI
File	File	PEI module	AcpiPlatformPei
File	File	PEI module	SmbusPei
File	File	PEI module	WdtAppPei
File	File	PEI module	PchInitPeim
File	File	PEI module	PchResetPeim
File	File	PEI module	PchSmbusArpDisabled
File	File	PEI module	PchSpiPeim
File	File	PEI module	PchUsb

Information

Type: 0x1B
Size: 0x0000D8
Parsed expression:
PUSH 30EB2979-B0F7-4D60-B2DC-1A2C96CEB1F4
PUSH B6EC423C-21D2-490D-85C6-DD5864EAA674
AND
PUSH 7408D748-FC8C-4EE6-9288-C4BEC092A410
AND
PUSH 3CDC90C6-13FB-4A75-9E79-59E9DD78B9FA
AND
PUSH 9A7EF41E-C140-48D1-B884-1E11240B4CE6
AND
PUSH ABD42895-78CF-4872-8444-1B5C180BF8DA
AND
PUSH 150CE416-EE63-46B6-8BA3-7322B8E04637
AND
PUSH F38D1338-AF7A-4FB6-91DB-1A9C2183570D
AND
PUSH 8C376010-2400-4D7D-B47B-9D851DF3C9D1
AND
PUSH EE0EA811-FBD9-4777-B95A-BA4F71101F74
AND
PUSH 3D0E663A-DC72-4489-87C5-E49EE773A452
AND
PUSH 09EA8911-BE0D-4230-A003-EDC693B48E11
AND
END

Messages

UEFI – Puntos de vista de atacante y defensor

- Cuanto código tan ordenadito. Además se puede programar en “C”.
- Formato PE...BIEN.
- A mi disposición el disco donde reside el S.O.
 - Existe módulo DXE NTFS
- Si puedo añadir mi driver DXE estoy en mejor situación que cuando modificaba el MBR/VBR... ¿NO?

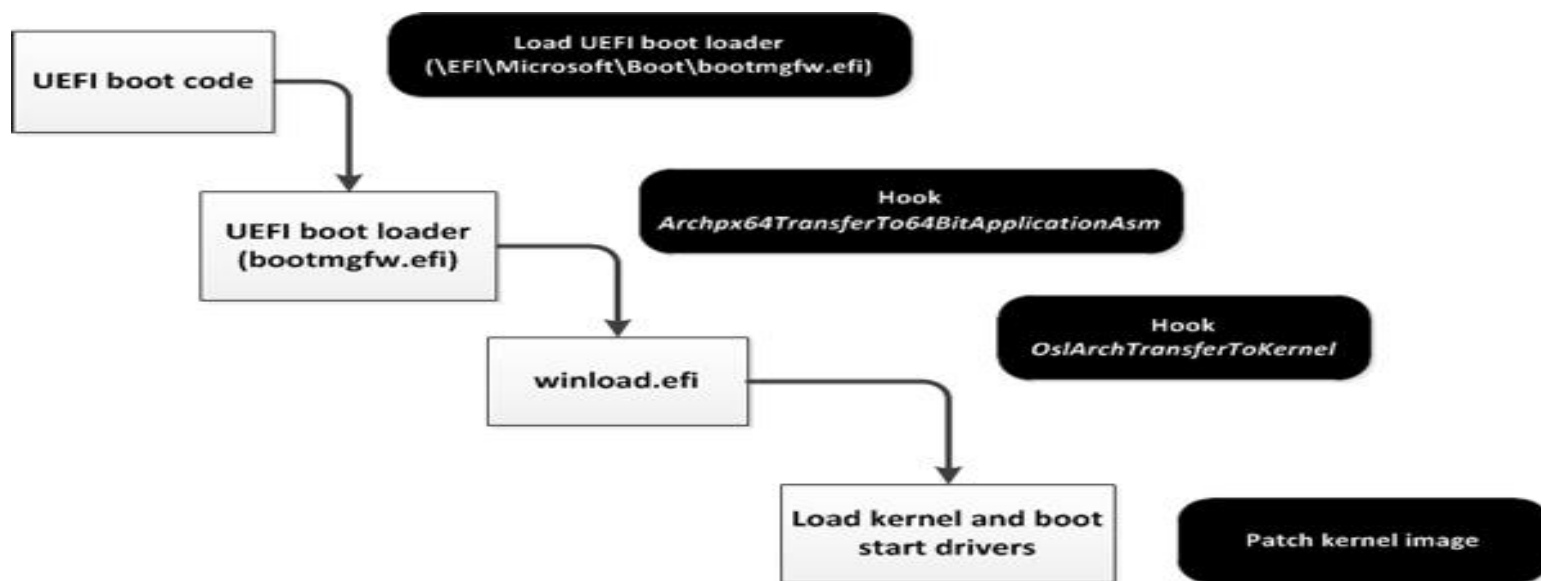


• ©1999 20TH CENTURY FOX FILM CORP.

- La estandarización es buena. Mil ojos velan por una especificación segura.
- Tengo conocimiento y control del SW instalado en mi FW
 - Puedo saber si ha habido modificaciones.
- Aunque el mantenimiento y gestión no es aún muy amigable, lo veo factible.

Una vez realizada la agresión, el camino es largo

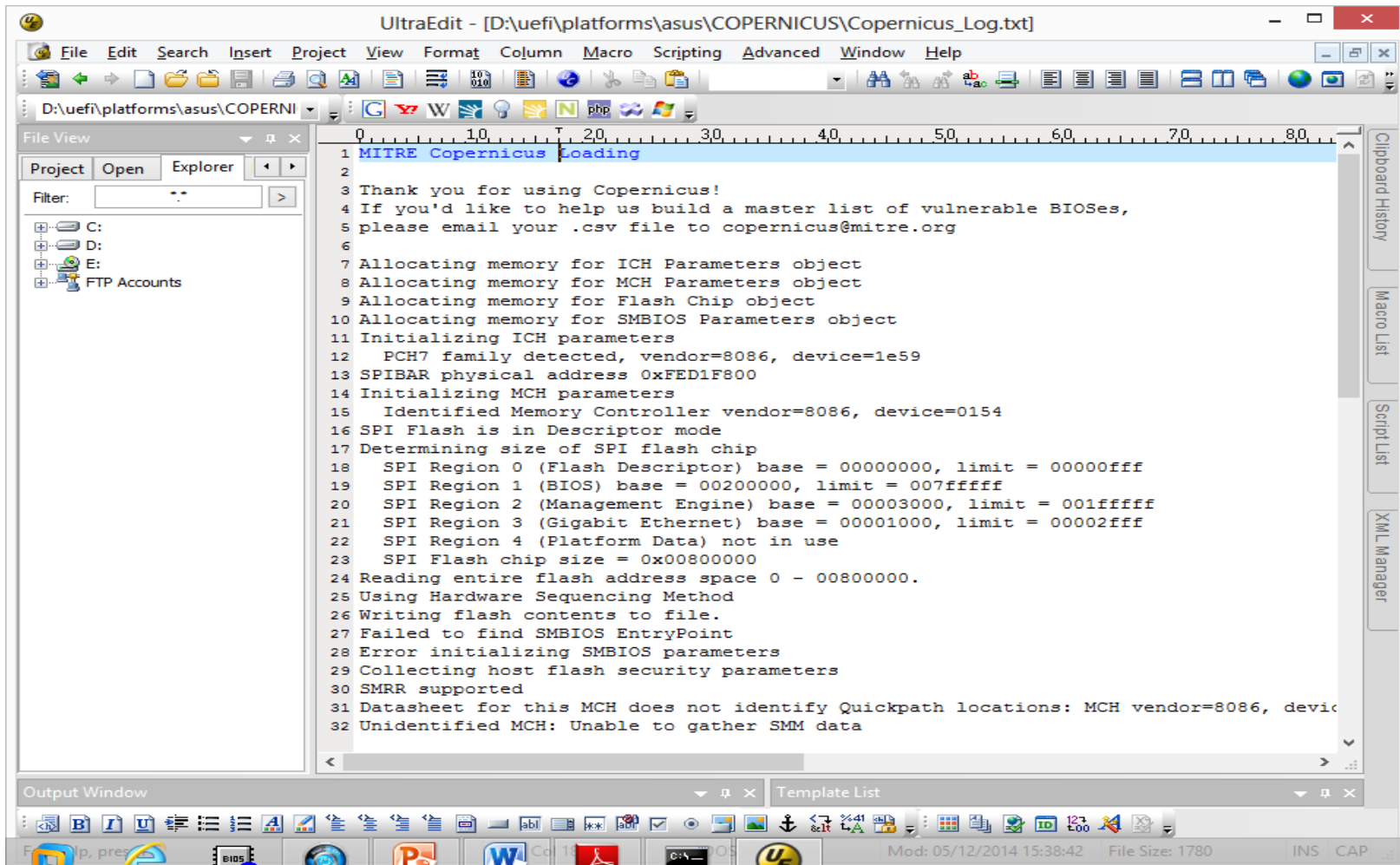
- Proceso de infección del Bootkit DreamBoot



sexy ...?



COPERNICUS – Herramienta de verificación de integridad y gestión de configuración



UltraEdit - [D:\uefi\platforms\asus\COPERNICUS\Copernicus_Log.txt]

File Edit Search Insert Project View Format Column Macro Scripting Advanced Window Help

D:\uefi\platforms\asus\COPERNI

File View

Project Open Explorer

Filter:

C:
D:
E:
FTP Accounts

```
1 MITRE Copernicus Loading
2
3 Thank you for using Copernicus!
4 If you'd like to help us build a master list of vulnerable BIOSes,
5 please email your .csv file to copernicus@mitre.org
6
7 Allocating memory for ICH Parameters object
8 Allocating memory for MCH Parameters object
9 Allocating memory for Flash Chip object
10 Allocating memory for SMBIOS Parameters object
11 Initializing ICH parameters
12   PCH7 family detected, vendor=8086, device=1e59
13 SPIBAR physical address 0xFED1F800
14 Initializing MCH parameters
15   Identified Memory Controller vendor=8086, device=0154
16 SPI Flash is in Descriptor mode
17 Determining size of SPI flash chip
18   SPI Region 0 (Flash Descriptor) base = 00000000, limit = 00000fff
19   SPI Region 1 (BIOS) base = 00200000, limit = 007fffff
20   SPI Region 2 (Management Engine) base = 00003000, limit = 001fffff
21   SPI Region 3 (Gigabit Ethernet) base = 00001000, limit = 00002fff
22   SPI Region 4 (Platform Data) not in use
23   SPI Flash chip size = 0x00800000
24 Reading entire flash address space 0 - 00800000.
25 Using Hardware Sequencing Method
26 Writing flash contents to file.
27 Failed to find SMBIOS EntryPoint
28 Error initializing SMBIOS parameters
29 Collecting host flash security parameters
30 SMRR supported
31 Datasheet for this MCH does not identify Quickpath locations: MCH vendor=8086, device=0154
32 Unidentified MCH: Unable to gather SMM data
```

Clipboard History
Macro List
Script List
XML Manager

Output Window

Template List

Mod: 05/12/2014 15:38:42 File Size: 1780 INS CAP

CHIPSEC – Platform Security Assessment Framework

Iniciativa de INTEL para verificar la seguridad del FW

Is BIOS correctly protected?

► common.bios_wp

```
[+] imported chipsec.modules.common.bios_wp
```

```
[x] [ =====
```

```
[x] [ Module: BIOS Region Write Protection
```

```
[x] [ =====
```

```
BIOS Control (BDF 0:31:0 + 0xDC) = 0x2A
```

```
[05] SMM_BWP = 1 (SMM BIOS Write Protection)
```

```
[04] TSS_ = 0 (Top Swap Status)
```

```
[01] BLE = 1 (BIOS Lock Enable)
```

```
[00] BIOSWE = 0 (BIOS Write Enable)
```

```
[+] BIOS region write protection is enabled (writes restricted to SMM)
```

```
[*] BIOS Region: Base = 0x00500000, Limit = 0x00FFFFFF
```

```
SPI Protected Ranges
```

PRx (offset)	Value	Base	Limit	WP?	RP?
PR0 (74)	00000000	00000000	00000000	0	0
PR1 (78)	8FFF0F40	00F40000	00FFF000	1	0
PR2 (7C)	8EDF0EB1	00EB1000	00EDF000	1	0
PR3 (80)	8EB00EB0	00EB0000	00EB0000	1	0
PR4 (84)	8EAF0C00	00C00000	00EAF000	1	0

```
[!] SPI protected ranges write-protect parts of BIOS region (other parts of BIOS can be modified)
```

```
[+] PASSED: BIOS is write protected
```


CONCLUSIONES I

- La seguridad tradicional todavía no ha puesto el foco en UEFI
- Un montón de código se ejecuta antes de que el S.O. tome el control
 - Este código tiene capacidad casi “ilimitada” sobre la plataforma hasta que se cede el control al S.O. Después casi NULA.
- La Integridad de los módulos en UEFI es a día de hoy un punto clave.
 - Control de las claves PK, KEK,...
 - Control e inventario de los módulos instalados.
- Si puedes grabar la NVRAM vía HW o SW
 - Si además están las claves PK, KEK, ...-> GAME OVER.
- Solo con el “abuso” de las propias características de la plataforma puede ser suficiente.
- Las carencias detectadas están en la implementación, no en la especificación
 - La complejidad de las plataformas y módulos adicionales
 - Los diferentes módulos que se arrancan en función del tipo de arranque (hibernación, suspensión, FastBoot ..) hacen mayor la superficie de ataque

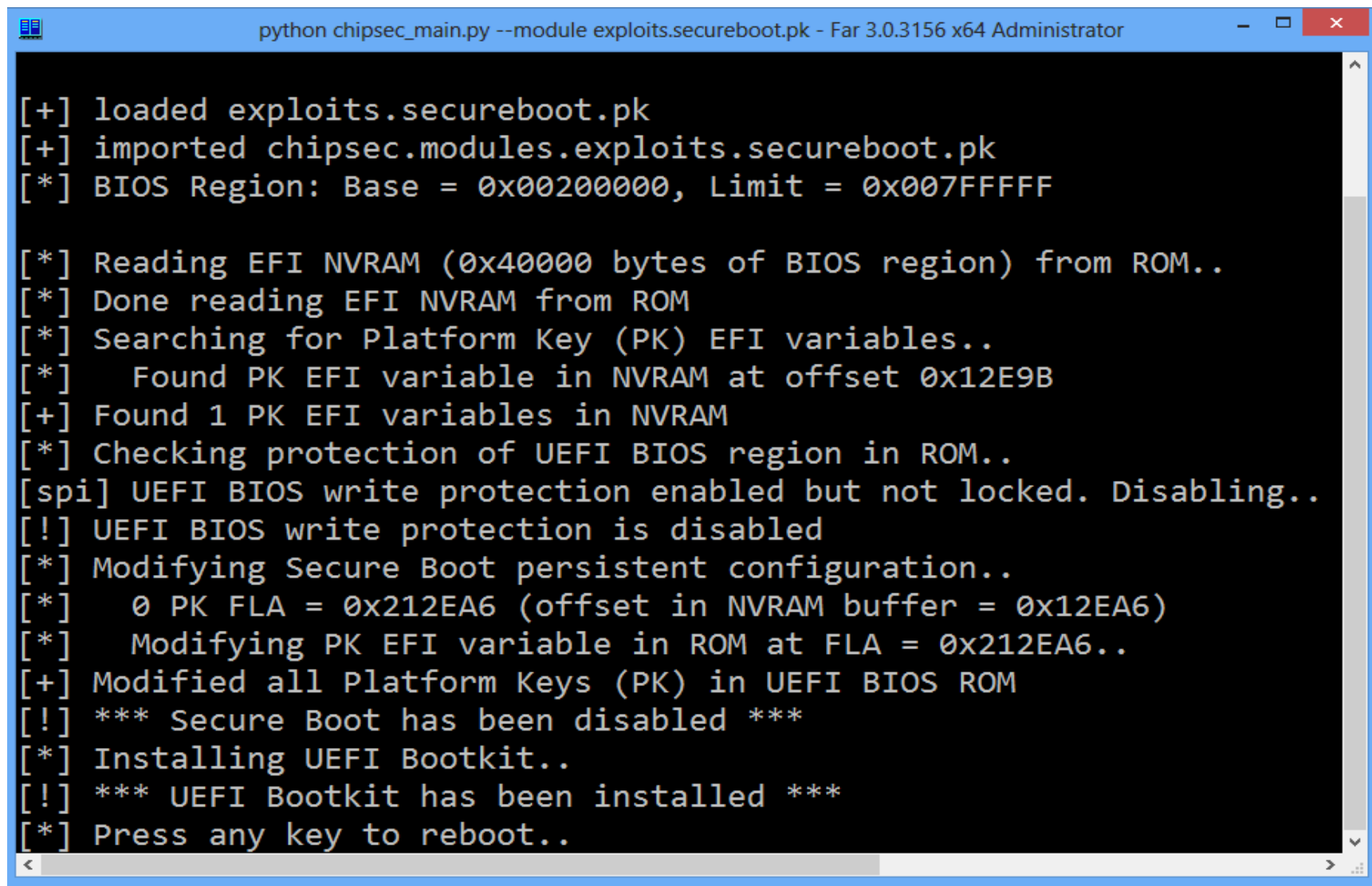
CONCLUSIONES II

- Ya existen POCs de “exploits” sobre código UEFI en el proceso de update.
- La utilización de herramientas como COPENICUS debe ayudar a gestionar la seguridad UEFI de un parque informático.
- La utilización del TPM para el almacenamiento de las claves ayudará a mejorar notablemente la implementación de SecureBoot.
- En el futuro inmediato es de esperar ver aplicaciones y drivers EFI implementando servicios de seguridad.
- Ya existen ataques viables. Su complejidad apunta a su utilización en objetivos escogidos. Ojo a “insiders”.

UEFI – UN ARMA DE DOBLE FILO



DEMO explicada



```
python chipsec_main.py --module exploits.secureboot.pk - Far 3.0.3156 x64 Administrator

[+] loaded exploits.secureboot.pk
[+] imported chipsec.modules.exploits.secureboot.pk
[*] BIOS Region: Base = 0x00200000, Limit = 0x007FFFFFFF

[*] Reading EFI NVRAM (0x40000 bytes of BIOS region) from ROM..
[*] Done reading EFI NVRAM from ROM
[*] Searching for Platform Key (PK) EFI variables..
[*]   Found PK EFI variable in NVRAM at offset 0x12E9B
[+] Found 1 PK EFI variables in NVRAM
[*] Checking protection of UEFI BIOS region in ROM..
[spi] UEFI BIOS write protection enabled but not locked. Disabling..
[!] UEFI BIOS write protection is disabled
[*] Modifying Secure Boot persistent configuration..
[*]   0 PK FLA = 0x212EA6 (offset in NVRAM buffer = 0x12EA6)
[*]   Modifying PK EFI variable in ROM at FLA = 0x212EA6..
[+] Modified all Platform Keys (PK) in UEFI BIOS ROM
[!] *** Secure Boot has been disabled ***
[*] Installing UEFI Bootkit..
[!] *** UEFI Bootkit has been installed ***
[*] Press any key to reboot..
```


Referencias (1)

☐ UEFI SPECS

☐ www.uefi.org

- BEYOND BIOS: Developing with the Unified Extensible Firmware Interface

☐ Vincent Zimmer et al

☐ SAFERBYTES IT SECURITY

☐ <http://news.saferbytes.it/analisi/2012/09/uefi-technology-say-hello-to-the-windows-8-bootkit/>

☐ CANSECWEST 2014

☐ Platform Firmware Security Assessment wCHIPSEC-csw14-final.pdf

☐ MICROSOFT TECHNET

☐ <http://technet.microsoft.com/en-us/library/hh824898.aspx>

Referencias (2)

- [1] Attacking Intel BIOS – Alexander Tereshkin & Rafal Wojtczuk – Jul. 2009
 - <http://invisiblethingslab.com/resources/bh09usa/Attacking%20Intel%20BIOS.pdf>
- [2] TPM PC Client Specification - Feb. 2013
 - http://www.trustedcomputinggroup.org/developers/pc_client/specifications/
- [3] Evil Maid Just Got Angrier: Why Full-Disk Encryption With TPM is Insecure on Many Systems – Yuriy Bulygin – Mar. 2013
 - <http://cansecwest.com/slides/2013/Evil%20Maid%20Just%20Got%20Angrier.pdf>
- [4] A Tale of One Software Bypass of Windows 8 Secure Boot – Yuriy Bulygin – Jul. 2013
 - <http://blackhat.com/us-13/briefings.html#Bulygin>
- [5] Attacking Intel Trusted Execution Technology - Rafal Wojtczuk and Joanna Rutkowska –
 - <http://invisiblethingslab.com/resources/bh09dc/Attacking%20Intel%20TXT%20-%20paper.pdf>
- [6] Another Way to Circumvent Intel® Trusted Execution Technology - Rafal Wojtczuk,
 - Joanna Rutkowska, and Alexander Tereshkin – Dec. 2009
 - <http://invisiblethingslab.com/resources/misc09/Another%20TXT%20Attack.pdf>
- [7] Exploring new lands on Intel CPUs (SINIT code execution hijacking) - Rafal Wojtczuk and Joanna Rutkowska - Dec. 2011
 - http://www.invisiblethingslab.com/resources/2011/Attacking_Intel_TXT_via_SINIT_hijacking.pdf

Referencias (3)

- [8] Implementing and Detecting an ACPI BIOS Rootkit – Heasman, Feb. 2006
 - <http://www.blackhat.com/presentations/bh-europe-06/bh-eu-06-Heasman.pdf>
- [9] Implementing and Detecting a PCI Rookit – Heasman, Feb. 2007
 - <http://www.blackhat.com/presentations/bh-dc-07/Heasman/Paper/bh-dc-07-Heasman-WP.pdf>
- [10] Using CPU System Management Mode to Circumvent Operating System Security Functions - Duflot et al., Mar. 2006
 - <http://www.ssi.gouv.fr/archive/fr/sciences/fichiers/liti/cansecwest2006-duflotpape.pdf>
- [11] Getting into the SMRAM:SMM Reloaded – Duflot et. Al, Mar. 2009
 - <http://cansecwest.com/csw09/csw09-duflot.pdf>
- [12] Attacking SMM Memory via Intel® CPU Cache Poisoning – Wojtczuk & Rutkowska, Mar. 2009
 - http://invisiblethingslab.com/resources/misc09/smm_cache_fun.pdf
- [13] Defeating Signed BIOS Enforcement – Kallenberg et al., Sept. 2013
 - http://www.syscan.org/index.php/download/get/6e597f6067493dd581eed737146f3afb/SyScan2014_CoreyKallenberg_SetupforFailureDefeatingSecureBoot.zip

Referencias (4)

- [14] Mebromi: The first BIOS rootkit in the wild – Giuliani, Sept. 2011
 - <http://www.webroot.com/blog/2011/09/13/mebromi-the-first-bios-rootkitin-the-wild/>
- [15] Persistent BIOS Infection – Sacco & Ortega, Mar. 2009
 - <http://cansecwest.com/csw09/csw09-sacco-ortega.pdf>
- [16] Deactivate the Rootkit – Ortega & Sacco, Jul. 2009
 - <http://www.blackhat.com/presentations/bh-usa-09/ORTEGA/BHUSA09-Ortega-DeactivateRootkit-PAPER.pdf>

➤ E-Mails

- ccn-cert@cni.es
- info@ccn-cert.cni.es
- ccn@cni.es
- sondas@ccn-cert.cni.es
- redsara@ccn-cert.cni.es
- carmen@ccn-cert.cni.es
- organismo.certificacion@cni.es

➤ Websites

- www.ccn.cni.es
- www.ccn-cert.cni.es
- www.oc.ccn.cni.es



Síguenos en Linked in